ChatGPT

Comprehensive Python Interview Questions (By Category)

Below is a categorized list of Python interview questions covering a range of topics. Each category is divided into basic, intermediate, and advanced questions to test different levels of proficiency. The content is up-to-date with Python 3.8–3.12 features and best practices.

1. Core Language Features

Basic Questions

- What are some **key features** of Python as a programming language (e.g., typing, memory management, syntax)?
- Is Python **interpreted or compiled**? Explain how Python executes code (hint: bytecode and the Python Virtual Machine) [1].
- What does it mean that Python is **dynamically typed** and **strongly typed**? How is this different from a statically typed language?

Intermediate Questions

- What is the difference between the `==` operator and the `is` operator in Python? When would two variables be `== True` but `is False`? [2]
- What is the **"walrus operator"** (`:=`) introduced in Python 3.8, and how does it improve certain expressions? [3]
- Why is it recommended to follow **PEP 8**, and can you give an example of a Pythonic coding style versus a non-Pythonic one?

Advanced Questions

- Python 3.10 introduced **structural pattern matching** (the `match` statement). What is pattern matching and how would you use `match`/`case` in practice? [4]
- In your own words, what does it mean for code to be *"Pythonic"*? Can you provide an example of a Pythonic solution to a problem versus a non-Pythonic one?

2. Data Types and Data Structures

Basic Questions

- What is the difference between a **list** and a **tuple** in Python? [5] Why might you use one over the other?
- How do you create a **dictionary** in Python, and what are some characteristics of dictionaries (e.g. key uniqueness, mutability)?
- What is a **set** in Python and how is it different from a list? When would you use a set instead of a list?

Intermediate Questions

- Explain **mutable vs. immutable** data types in Python. Which built-in types are immutable, and why is immutability important (for example, for dict keys)?
- When would you use a **list** vs. a **set** in terms of performance or semantics? (Consider operations like look-up membership and maintaining order.)
- What does it mean for dictionary keys to be **hashable**? Give examples of types that can and cannot be used as dict keys.

Advanced Questions

- What is the difference between a **shallow copy** and a **deep copy** of an object in Python? In what scenarios could using one or the other lead to bugs? [6]
- How are Python **lists** implemented under the hood? Discuss how they manage capacity and how operations like `append` and `insert` perform in terms of Big-O complexity.
- Describe how Python's **dictionary** is implemented. What is the underlying data structure, and how does Python handle hash collisions or maintain insertion order (as of Python 3.7+)?

3. Control Flow and Functions

Basic Questions

- When would you use a `for` **loop** versus a `while` **loop** in Python? Give an example of each.
- Explain the purpose of the `break` and `continue` statements inside loops. How do they alter loop execution?
- How do you write a **ternary conditional** (one-line if-else) in Python, and when might it be useful?

Intermediate Questions

- What are `*args` and `**kwargs` in Python function definitions? How do you use them, and when are they helpful?
- Explain how default function arguments work. What is the "**mutable default argument**" pitfall, and how can you avoid it (e.g. with a sentinel value)?
- Python allows an `else` clause after loops. When does the `else` **block on a loop** execute, for `for` or `while` loops?

Advanced Questions

- What is a **closure** in Python? Can you demonstrate with an example how an inner function can "remember" variables from its enclosing scope?
- How does Python's **LEGB (Local, Enclosed, Global, Built-in)** scoping rule determine variable resolution? For instance, what happens if you try to modify a global variable inside a function?
- Python's `for` and `while` loops support an `else` clause. Can you explain a scenario where a `loop...else` is useful and how it works (especially when combined with `break`)?

4. Object-Oriented Programming (OOP)

Basic Questions

- What is a **class** in Python, and how do you create an object (instance) from a class?

- Explain the difference between a **class attribute** and an **instance attribute**. How would changing one affect all instances vs. just one instance?
- What is the role of the `__init__` method in a Python class, and what is the purpose of the `self` parameter?

Intermediate Questions

- Does Python support **multiple inheritance**? If yes, how does it handle methods or attributes that appear in multiple parent classes? [7]
- What is **polymorphism** in the context of OOP, and how does Python achieve polymorphism? (Consider duck typing or method overriding in subclasses.)
- Explain the difference between an **instance method**, a **class method**, and a **static method** in Python. When would you use `@classmethod` or `@staticmethod`? [8] [9]
- What are **dataclasses** (the `dataclasses` module introduced in Python 3.7), and what benefits do they provide when defining classes? [10]

Advanced Questions

- What is the **method resolution order (MRO)** in Python, and how does Python determine which method to call first when dealing with multiple inheritance?
- Python 3.12 added a new decorator `typing.override` (PEP 698) for indicating method overrides. What does the `@override` decorator do, and how is it meant to be used in class design? [11]
- What is a **metaclass** in Python? How would you define one, and in what situations might using a metaclass be appropriate?

5. Functional Programming

Basic Questions

- What is a **lambda function** in Python? How does a lambda differ from a normal function, and can you give an example of using one?
- How do the built-in functions `map()`, `filter()`, and `reduce()` work? What does each do, and how are they related to the functional programming paradigm?
- Python supports **list comprehensions**. How is a list comprehension similar to or different from using `map` or a `for` loop to build a list?

Intermediate Questions

- What is a **higher-order function**? Can you give an example of treating a function as a first-class object in Python (such as passing a function into another function)?
- How do **generator expressions** differ from list comprehensions? What benefits do they offer in terms of performance or memory usage?
- Explain how functions like `filter` or `sorted` can accept another function (like a lambda) as an argument. How does this ability fit into Python's support for functional programming?

Advanced Questions

- What is **currying** or **partial function application**, and how can you achieve it in Python? (Hint: `functools.partial`.) When might you use this technique?
- Python's `functools` module provides an `lru_cache` **decorator**. What does this decorator do, and how can it be used to *memoize* (cache) function results for optimization?

- Discuss the concept of **immutability** in functional programming. Does Python encourage immutability? Give examples of writing functions without side effects in Python.

6. Modules and Packaging

Basic Questions

- What is a **module** in Python, and how do you use the `import` statement to include code from one module in another?
- What is a **package** in Python? How does an `__init__.py` file relate to defining a package?
- How can you import a module using an **alias** (e.g. `import numpy as np`), and why might you do that?

Intermediate Questions

- When you execute an `import` statement, where does Python look for modules? Describe how `sys.path` and the **PYTHONPATH** environment variable affect module searching [12].
- What are **relative imports** in Python packages, and when would you use them versus absolute imports?
- Explain how you would organize a **large Python project** into modules and packages. What factors influence how you decide to structure the code into separate files or namespaces?

Advanced Questions

- What are **namespace packages** (as defined in PEP 420)? How do they differ from regular packages, and why might you use one?
- How would you create a **distributable Python package** (one that can be uploaded to PyPI)? Outline the purpose of files like `setup.py` or `pyproject.toml` in packaging.
- If you wanted to share code between multiple projects without copy-pasting, how could you turn that code into a **reusable module or library**? (Consider packaging, installing via pip, etc.)

7. File I/O and OS Operations

Basic Questions

- How do you **open a file** in Python for reading or writing? Demonstrate with an example of reading all lines from a text file (using Python's context manager).
- What is the advantage of using a `with open(...) as f:` block when dealing with file I/O?
- How can you check if a file or directory **exists** on the filesystem using Python? Which module provides such utilities?

Intermediate Questions

- How do you **list all files** in a directory in Python? Name two different approaches or modules you could use.
- What is the difference between opening a file in **text mode** vs **binary mode**? When would you use `'rb'` or `'wb'` as the mode for `open()`?
- How can you **create** and **remove** directories in Python via script? Which modules and functions would you use?

Advanced Questions

- When dealing with very large files, how can you read the file **efficiently** without loading it entirely into memory at once?
- How would you **execute a shell command** from within a Python program? (For example, calling an external program or script and capturing its output.)
- Explain how you might implement **file locking** or safe writes to a file to prevent race conditions when multiple processes could write to the same file.

8. Exception Handling and Debugging

Basic Questions

- How do you handle **exceptions** in Python? Give a simple example using a `try/except` block around code that might raise an error.
- What is the purpose of the `finally` clause in exception handling? When is the code inside a `finally` block executed?
- Can you have an `else` block as part of a try-except structure? If so, when does the `else` part run in a `try/except/else` construct?

Intermediate Questions

- How can you raise a **custom exception** in Python? Write a small example of defining your own exception class and raising it.
- Why should you avoid using a bare `except:` clause? How can catching overly broad exceptions be problematic?
- Suppose you catch an exception but want to **re-raise** it (or part of it) to add context. How do you use the `raise ... from ...` syntax to chain exceptions?

Advanced Questions

- Python 3.11 introduced **Exception Groups** (PEP 654) allowing multiple exceptions to be raised together. What is an **ExceptionGroup**, and how do you handle such grouped exceptions with the new `except*` syntax? [13]
- How would you go about **debugging** a Python program that is crashing without an obvious error? (For example, what tools or techniques could you use – think of the `logging` module, `pdb`, print debugging, etc.)
- Describe how the Python interpreter prints a **traceback** when an unhandled exception occurs. What information does the traceback give you, and how can it be useful in debugging?

9. Iterators, Generators, and Comprehensions

Basic Questions

- What is a **list comprehension** in Python? Provide an example of using a list comprehension to create a list of squares of numbers 1–5.
- How does a **generator expression** (`(x for x in ...)`) differ from a list comprehension `[...]`? What do you get when you evaluate a generator expression?
- Give an example of a basic **generator function** using the `yield` keyword. What does calling this generator function return?

Intermediate Questions

- What is the **iterator protocol** in Python? Describe what methods an object needs to implement to be considered an iterator [14].
- Explain how the `for` loop works internally with iterators. What does Python call on the iterable to get an iterator, and how does it retrieve values from the iterator?
- How do **generator functions** differ from regular functions in terms of execution and state? What happens when you call `yield` inside a function?

Advanced Questions

- How would you create a **custom iterable** class in Python? Outline what methods you would implement so that you can loop over instances of your class (e.g., defining `__iter__` and `__next__`).
- Can you compare **generator functions** and **generator expressions** in terms of memory usage and readability? When might you choose one over the other?
- Python 3.12 includes an optimization PEP 709 (inlined comprehensions). Generally, why are list/set/dict **comprehensions** often faster than equivalent manual loops? (Consider Python's implementation details.)

10. Decorators and Context Managers

Basic Questions

- What is a **decorator** in Python? How do you define a simple decorator that, for example, logs when a function is called?
- Show how you would apply a decorator to a function using the `@` syntax. What exactly happens (behind the scenes) when you write `@my_decorator` above a function definition?
- Give an example of a scenario where using a decorator is useful (e.g., modifying or checking function arguments, caching results, etc.).

Intermediate Questions

- What is a **context manager** in Python? How do the `__enter__` and `__exit__` methods work to set up and tear down context?
- Python's `with` statement is often used with context managers (for example, `with open('file.txt') as f:`). What benefits does using `with` provide?
- Name a few examples of context managers in the standard library (or in common frameworks) and explain what resources they manage.

Advanced Questions

- How would you implement a custom context manager *without* using any helper decorators? Write a sketch of a class that implements `__enter__` and `__exit__` to manage some resource.
- The `contextlib` module provides a `@contextmanager` decorator. How does using `@contextmanager` and a generator function simplify creating a context manager?
- When writing a decorator, why is using `functools.wraps` important? What problems does it solve in terms of the decorated function's metadata?

11. Concurrency, Multithreading, and Multiprocessing

Basic Questions

- What is the difference between a **thread** and a **process**? How does Python's threading relate to the operating system's threads?
- Explain the concepts of **concurrency vs. parallelism** in general. Can you have concurrency without true parallel CPU execution in Python?
- What is a **race condition**? Can you give a simple example in Python (or pseudocode) where a race condition might occur?

Intermediate Questions

- What is the **Global Interpreter Lock (GIL)** in CPython, and how does it affect multi-threaded Python programs? What kinds of tasks are most impacted by the GIL?
- When would you choose to use **multithreading** in Python, and when would you choose **multiprocessing**? Provide an example scenario for each.
- How can you safely share data between threads or processes? Mention tools like thread locks (mutexes) for threads or IPC mechanisms for processes.

Advanced Questions

- What is **asyncio** and the `async/await` syntax in Python? How is asynchronous programming different from threading, and what kind of concurrency does it achieve?
- Python 3.12 implemented PEP 684, introducing a **per-interpreter GIL** for subinterpreters [15]. What is the idea behind a per-interpreter GIL, and how could it potentially improve Python's ability to utilize multiple CPU cores?
- Describe how you would design a Python program to handle **many simultaneous network requests** (for example, a chat server). What combination of concurrency techniques (threads, async, processes) might you use and why?

12. Testing and Mocking (unittest, pytest)

Basic Questions

- How do you write a simple **unit test** in Python using the `unittest` framework? Outline the steps (creating a subclass of `unittest.TestCase`, writing test methods, running them).
- What is the purpose of **assertions** in testing? Give an example of an assertion you might use in a test (either using `unittest` methods like `assertEqual` or the `assert` statement in general).
- Have you used the `pytest` framework? If so, what are some advantages of pytest over unittest (for example, in test discovery or assertion reporting)?

Intermediate Questions

- What is **mocking** in the context of testing? When would you use something like `unittest.mock.patch` or `MagicMock` in your tests?
- How can you test that a piece of code **raises an exception** as expected? (For example, using `with self.assertRaises` in unittest or `pytest.raises` in pytest.)
- Explain what a **test fixture** is. In `unittest`, how do `setUp` and `tearDown` methods work? In `pytest`, how do you achieve similar setup/teardown of test state?

Advanced Questions

- In pytest, what are **fixtures** and how do they improve test design? How do you create a fixture and use it in multiple test functions (mention the `@pytest.fixture` decorator or fixture autouse)?
- How would you use the `unittest.mock` library to **simulate an external dependency** (like a database or network call) so that your unit tests run quickly and reliably?
- Discuss strategies for organizing a **large test suite** in a Python project. How do you ensure tests remain fast, isolated, and maintainable as the codebase grows?

13. Virtual Environments and Packaging (pip, venv, poetry)

Basic Questions

- What is a Python **virtual environment** and why is it useful to use one? How do you create a virtual environment using the built-in `venv` module?
- Explain the role of `pip` in Python. How do you use pip to install a package, and where does pip typically install packages?
- What is a **requirements.txt** file and how is it used in Python projects?

Intermediate Questions

- What problem do tools like **Pipenv or Poetry** solve beyond what pip offers? Have you used any of these, and how do they differ from plain pip and `requirements.txt`?
- How do you specify **dependencies** for your Python project? Describe one modern way to declare dependencies (for example, in a `pyproject.toml` or `setup.cfg`).
- What is the difference between **development dependencies** and **production (runtime) dependencies**, and how might you handle them in project configuration?

Advanced Questions

- What is a Python **wheel** file (`.whl`), and how does it differ from a source distribution (`sdist`)? What are the advantages of providing wheels for your package? [16]
- Have you heard of PEP 517/518 and the move to `pyproject.toml` for building packages? In general, how is modern Python packaging evolving compared to the older `setup.py` approach?
- Imagine you want to publish a library to PyPI. What are the steps to **package and upload** your project (mention build tools like `build` or `setuptools`, Twine for upload, etc.)?

14. Performance Optimization and Memory Management

Basic Questions

- What tools or methods can you use to **profile** the performance of Python code? (For example, have you used the `timeit` module or the `cProfile` profiler?)
- Explain Big-O **time complexity** in the context of Python data structures. For instance, what is the complexity of looking up an element in a list vs a dictionary?
- How would you roughly measure the **memory usage** of a Python object or a program at runtime?

Intermediate Questions

- How does Python manage **memory** for objects? Describe the concept of **reference counting** and how the garbage collector deals with reference cycles [17].
- What is a **memory leak**, and can memory leaks happen in Python despite garbage collection? Give an example scenario (perhaps involving lingering references or object caches).
- How could you profile which parts of your code are using the most CPU or memory? (Think of modules like `cProfile` for CPU, or `tracemalloc` for memory allocation tracking.)

Advanced Questions

- The Global Interpreter Lock can be a bottleneck for CPU-bound programs. What are some ways to **optimize CPU-bound tasks** in Python despite the GIL (e.g., using multiprocessing, native extensions, NumPy for vectorization, etc.)?
- What are some techniques to **optimize Python code** for speed? For example, when might you use list comprehensions, generator expressions, or built-in functions to gain performance?
- Explain the purpose of the `__slots__` declaration in a class. How does using `__slots__` potentially **reduce memory usage**, and what are the trade-offs of using it?

15. Python Internals (GIL, bytecode, CPython, etc.)

Basic Questions

- What is **CPython**? How is it related to the Python language, and what are some other Python implementations you know (e.g., PyPy, Jython)?
- In Python, what happens to your code when you run a .py file? Describe the steps from source code to execution (tokenizing, compiling to bytecode, interpreting, etc.).
- How can you inspect the **bytecode** of a Python function? (For instance, which module can disassemble Python bytecode?)

Intermediate Questions

- Name a couple of **alternative Python interpreters** or runtimes (such as PyPy, Jython, IronPython). What is a key difference one of these has compared to CPython?
- How does Python implement **garbage collection** in addition to reference counting? (Hint: think about how it detects and breaks reference cycles.)
- What are **Python's built-in types implemented in** (e.g., list, dict)? Discuss how Python's C implementation of these structures contributes to performance.

Advanced Questions

- Walk through what happens internally when you execute `import some_module`. How does Python *find* the module, and what steps does it take to load and initialize it? [12]
- What is the **Global Interpreter Lock** (GIL) and *why* was it originally introduced in Python's implementation? (Consider memory management and thread safety in CPython.)
- There's a proposal (PEP 703) to make a **GIL-free** or no-GIL build of CPython. What would removing the GIL entail, and why is it a challenging change for Python's runtime?

16. Web Development (Flask, Django)

Basic Questions

- What are the main differences between **Flask** and **Django** as Python web frameworks? When might you choose Flask over Django and vice versa?
- In a Flask application, how do you define a **route** for a URL endpoint? Provide a short example of a basic Flask route handling a GET request.
- What does Django's **MTV** (Model-Template-View) or MVC pattern mean? Can you briefly describe the roles of the Model, Template, and View in Django?

Intermediate Questions

- How do you define a database **model** in Django? Give a simple example of a model class and explain what Django does with that model (migrations, ORM queries).
- What is an **ORM (Object-Relational Mapper)** and how does Django's ORM help you interact with the database without writing SQL?
- How would you handle **form validation** in a web app? Compare how Flask might handle form data vs. how Django's forms or ModelForms work.

Advanced Questions

- What is **WSGI** in the context of Python web applications, and what role does it play in deploying a Flask/Django app?
- Django has support for **ASGI** to allow asynchronous views (and things like WebSockets via Django Channels). Explain the difference between WSGI and ASGI in terms of request handling [18].
- How would you **scale a Django web application** to handle increased load? Discuss techniques like using a caching layer, database optimization, load balancing with Gunicorn/uWSGI, etc.

17. Data Science (NumPy, Pandas, Matplotlib)

Basic Questions

- What is **NumPy** and how does a NumPy array differ from a regular Python list in terms of functionality and performance?
- Explain what a **pandas DataFrame** is. How would you create a DataFrame and perform a simple operation like filtering rows or selecting a column?
- How do you create a basic **plot** using Matplotlib? For example, how would you plot a line graph of a list of values?

Intermediate Questions

- What does it mean to **vectorize** operations in NumPy, and why is it beneficial for performance?
- How would you handle **missing data** in a pandas DataFrame? (Discuss methods like dropping missing values vs. filling them with imputations.)
- Explain the purpose of **Matplotlib** vs **Seaborn** vs **Plotly** libraries. When might you choose one plotting library over the others?

Advanced Questions

- What is **broadcasting** in NumPy and how does it work? For instance, how can NumPy add a 1-dimensional array to every row of a 2D array without explicit loops? [19]
- In pandas, what are some differences between a **Series** and a **DataFrame**? When would you end up with a Series versus a one-column DataFrame, and why does it matter?
- If you have to process a dataset that's larger than memory, what strategies or tools in the Python data science ecosystem could you use (e.g., chunked processing with pandas, using Dask for distributed DataFrames, etc.)?

18. Machine Learning and AI (scikit-learn, TensorFlow, etc.)

Basic Questions

- What is **scikit-learn** and what kinds of machine learning tasks is it used for? How is that different from **TensorFlow** or **PyTorch**, and when would you use each?
- Describe the typical **workflow of training a machine learning model** using scikit-learn. What are the main steps from preprocessing data to evaluating the model?
- In the context of machine learning, what is **overfitting** and how can you prevent it? (You can mention techniques like cross-validation, regularization, etc.)

Intermediate Questions

- How do you save a trained scikit-learn model for later use? (For example, which Python modules or functions might you use to serialize the model.)
- What is a **hyperparameter** versus a model parameter? Can you give examples of hyperparameters for a decision tree or neural network, and how you might tune them?
- Explain the difference between **supervised** and **unsupervised** learning. Provide an example algorithm from scikit-learn for each category and what it does.

Advanced Questions

- What is a **computational graph** in TensorFlow? Contrast how model execution worked in TensorFlow 1.x (graph mode) with how it works in TensorFlow 2.x with eager execution [20].
- How does **TensorFlow** leverage GPUs for acceleration? What kind of operations in deep learning benefit most from GPU parallelism, and how do you ensure your code runs on the GPU?
- Discuss how you would **deploy a machine learning model** in a production environment. What are some strategies or tools for serving predictions from a trained model (e.g., using TensorFlow Serving, Flask API, AWS SageMaker)?

19. Scripting and Automation

Basic Questions

- In a Python script, why would you include the idiom `if __name__ == "__main__":`? What does this guard do, and what are the benefits of using it?
- How can you make a Python script **executable** from the command line (on Unix-based systems)? (Hint: think about shebang lines and file permissions.)
- Which module would you use to quickly **print output in color** to the terminal or to prompt for user input? (This is to gauge knowledge of basic I/O and possibly libraries like `colorama` or `click` for CLI utils.)

Intermediate Questions

- How can you **parse command-line arguments** in a Python program? Explain how you'd use the `argparse` module to handle options and flags.
- Give an example of using the `subprocess` module to **run an external command** (like calling `ls` or launching another script) from within Python. How do you capture the output of that command?
- What does the `sched` or `time` module do? Can you outline a simple way to have a Python script perform a task on a schedule or after a delay (besides just using cron)?

Advanced Questions

- Suppose you need to automate interactions with a website (filling forms, clicking buttons). What Python tools or libraries could help you achieve that (mention things like **Selenium** or **requests** + BeautifulSoup for web scraping)?
- How would you implement a simple **worker queue** in Python for automation tasks? (For example, using a library like Celery, or even just threading with a queue to manage background tasks.)
- Describe how you might package a Python script as a **command-line tool** that others can install (using `setuptools` entry points or `pipx`) so that it can be run globally by name.

20. System Design and Architecture with Python

Basic Questions

- What is a **design pattern** in software development? Can you name a couple of design patterns and briefly describe how you might implement one in Python (for example, Singleton, Factory, Observer)?
- How would you structure a **large Python application** for maintainability? Talk about how you'd break the app into modules or packages and manage things like configuration, logging, etc.
- What are some important considerations when writing **readable and maintainable** Python code, especially as the codebase grows? (Think about style guides, documentation, modular design.)

Intermediate Questions

- What is a **microservices** architecture, and do you think Python is a good choice for building microservices? What libraries or frameworks might you use to create a microservice (e.g., Flask/FastAPI for an API server)?
- How would you design a **RESTful API** using Python? Outline the components you'd need (routing, request handling, data serialization, authentication, etc.) and mention any frameworks that simplify this (like Flask, Django REST Framework, FastAPI).
- In deploying Python applications, what are some ways to ensure **reliability and scalability**? (Consider concepts like using Docker containers, load balancers, process managers like Gunicorn, or cloud services.)

Advanced Questions

- You're tasked with building a **highly scalable web service** in Python (say, to handle thousands of requests per second). What architectural patterns and tools would you use? Discuss aspects like caching (Redis/Memcached), database scaling, asynchronous workers, etc.

- How would you integrate a Python service into a larger system that has services written in other languages? Discuss using **REST APIs, message queues (RabbitMQ, Kafka)**, or RPC mechanisms (gRPC) for communication.
- Describe a time (or scenario) where you would choose **Python for a system design** despite performance concerns. How would you mitigate Python's performance overhead at the architecture level (for example, by moving certain components to faster languages, using caching, or scaling horizontally)?

---

[1] [6] [7] [8] [9] [17] Python Interview Questions and Answers - GeeksforGeeks
https://www.geeksforgeeks.org/python/python-interview-questions/

[2] Understanding the Difference Between == and is in Python | by Javascript Jeep | Medium
https://jagathishsaravanan.medium.com/understanding-the-difference-between-and-is-in-python-d4c62295b277

[3] python - What are assignment expressions (using the "walrus" or ":=" operator)? Why was this syntax added? - Stack Overflow
https://stackoverflow.com/questions/50297704/what-are-assignment-expressions-using-the-walrus-or-operator-why-was-t

[4] Structural pattern matching in Python 3.10
https://benhoyt.com/writings/python-pattern-matching/

[5] Crush Your Python Interview: 16 Technical Questions Explained with Code | by From Code to Clarity | Jul, 2025 | Medium
https://medium.com/@oz3dprinter/crush-your-python-interview-16-technical-questions-explained-with-code-53d992d810a6

[10] Why You Should Use Python Data Classes - DEV Community
https://dev.to/omrigm/why-you-should-use-python-data-classes-48po

[11] [15] What's New In Python 3.12 — Python 3.13.5 documentation
https://docs.python.org/3/whatsnew/3.12.html

[12] Python Imports: A Guide to Packages, Modules, and Paths | by Gaurav Garg | Python in Plain English
https://python.plainenglish.io/python-imports-a-guide-to-packages-modules-and-paths-d51561bf56bb

[13] What's New In Python 3.11 — Python 3.13.5 documentation
https://docs.python.org/3/whatsnew/3.11.html

[14] Python Iterators
https://www.w3schools.com/python/python_iterators.asp

[16] Learn about Building a Python Package — Python Packaging Guide
https://www.pyopensci.org/python-package-guide/package-structure-code/python-package-distribution-files-sdist-wheel.html

[18] Difference Between ASGI and WSGI in Django - GeeksforGeeks
https://www.geeksforgeeks.org/python/difference-between-asgi-and-wsgi-in-django/

[19] NumPy Array Broadcasting - GeeksforGeeks
https://www.geeksforgeeks.org/numpy/numpy-array-broadcasting/

[20] Introduction to graphs and tf.function | TensorFlow Core
https://www.tensorflow.org/guide/intro_to_graphs